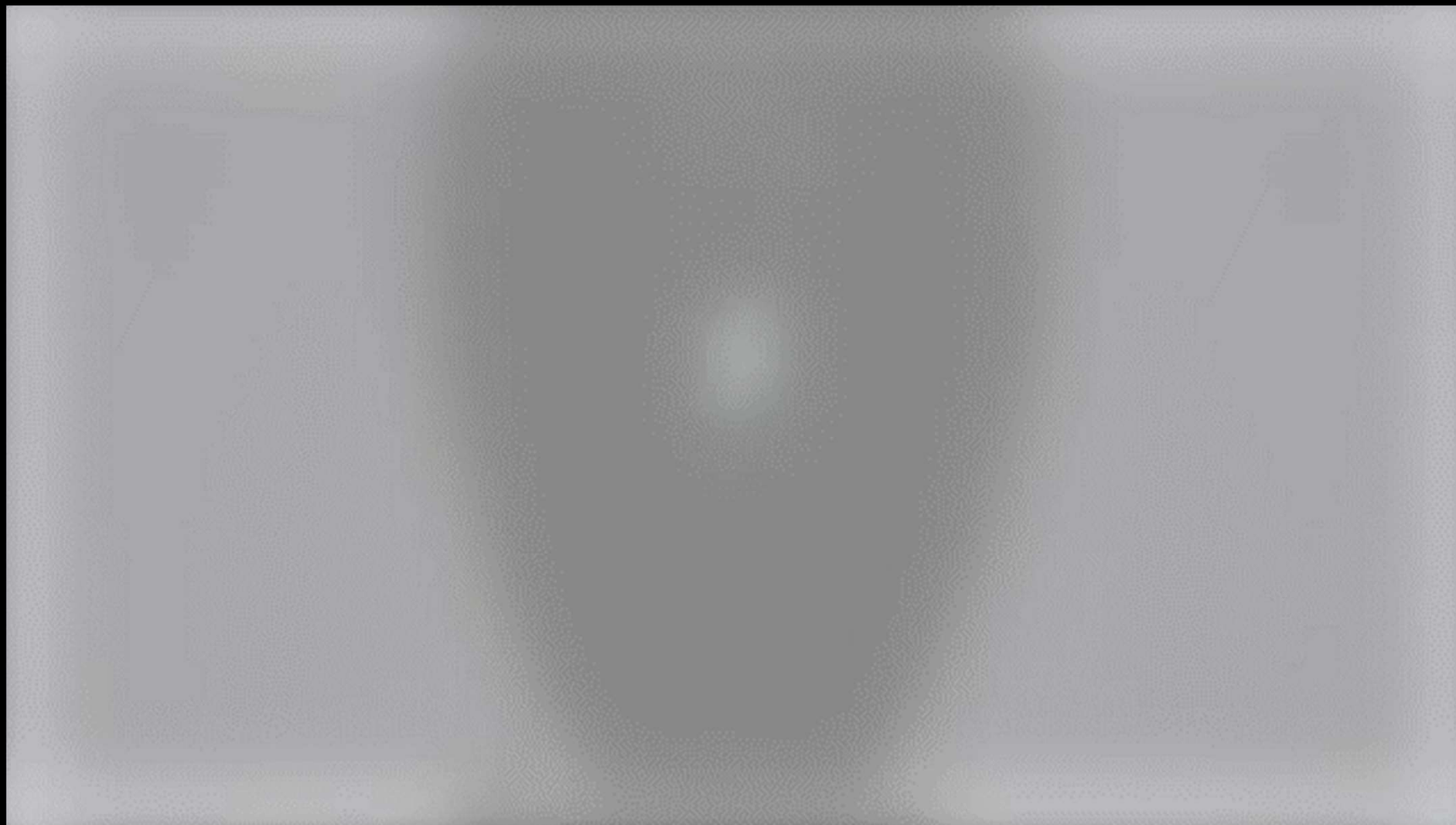


# 모방학습 개념 정리



**Figure, Figure Status Update - OpenAI Speech-to-Speech Reasoning**  
<https://www.youtube.com/watch?v=Sq1QZB5baNw>

A black and white photograph of two humanoid robots in a bedroom. The robots are white with black joints and faces. They are standing on either side of a bed, which has a striped blanket. The robot on the right is holding a small object in its hand. The background shows a window with multiple panes.

# HELIX

## BEDROOM TIDY

## 왜 휴머노이드에 열광할까?

“세상이 인간 기준으로 설계되어 있어서”

문 손잡이, 서랍, 스위치, 공구, 작업대 높이, 계단/문턱, 안전 규정, 작업 동선...

환경을 바꾸는 비용이 매우 크기 때문에, 로봇이 인간 환경에 맞춰야 하는 경우가 많다.

‘범용성’이 목표일 때 가장 강력한 품팩터

특정 라인이나 특정 작업에 최적화된 로봇은 빠르게 성공하지만, 작업이나 장소가 바뀌면 다시

지그나 센서구성 등 환경 구축이 필요하고, 안전분석이 필요하다.

이에 비해, 휴머노이드는 ‘바뀌는 작업’에 적응하는 범용 플랫폼으로 설계되기 쉽다.

“ 휴머노이드가 “멋있어서”가 아니라, 인간 환경에서의 범용 작업을 위해 필요한 ‘종합 패키지’이기 때문 ! ”



에이로봇의 ALICE



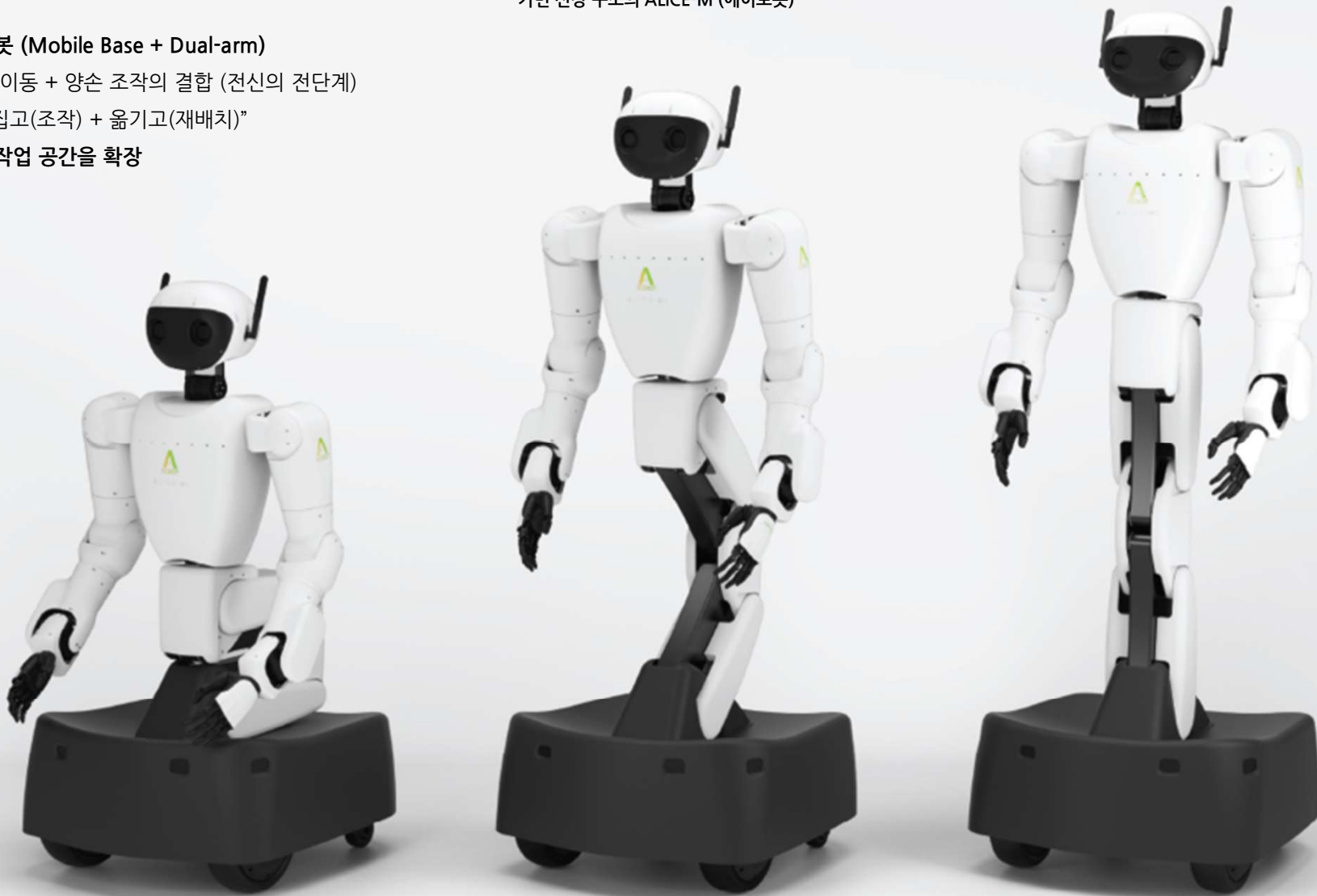
모바일로봇 + 양팔로봇 (Mobile Base + Dual-arm)

휴머노이드 요소 중.. 이동 + 양손 조작의 결합 (전신의 전단계)

핵심: “가서(이동) + 집고(조작) + 옮기고(재배치)”

강점: 인간 환경에서 작업 공간을 확장

가변 신장 구조의 ALICE-M (에이로봇)

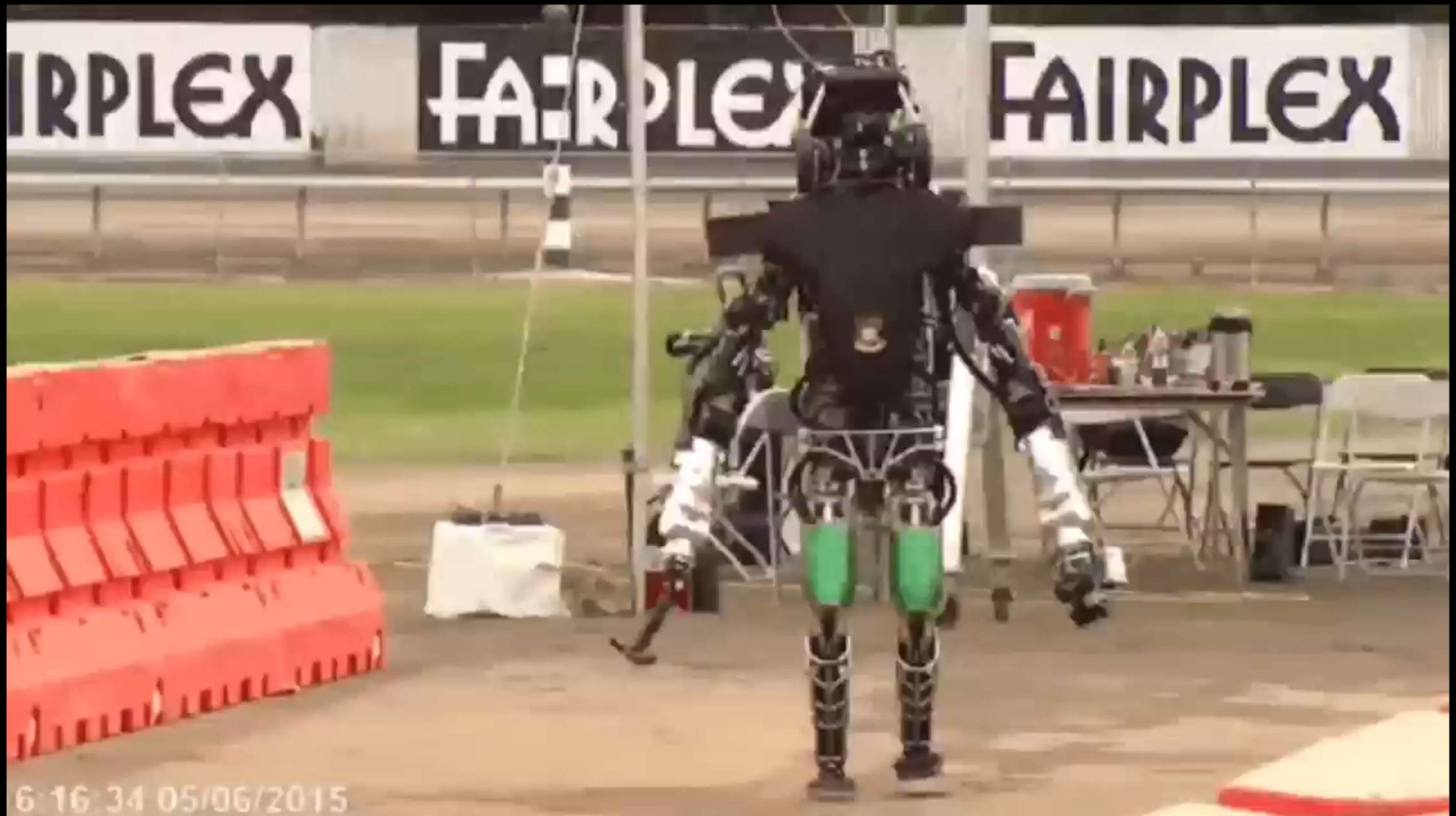


모방학습, 꼭 해야하나?

# 전통적인 모델 기반의 한계

- 전통적인 모델 기반의 로봇팔 제어도 “잘 되는 영역이 분명히 있다”
- 작업이 “고정된 환경+정해진 물체”일 때는 최적
- 정확한 목표 자세 및 궤적 추종
  - : PID, Computed Torque, 임피던스 및 어드미턴스 제어
- 계획 기반 접근
  - : IK, 경로계획, MPC
- 산업 현장 반복 작업
  - : 픽애플레이스, 용접, 도장, 조립 일부

하지만 현실 작업이 바뀌면서 한계가 커졌다..  
“ 접촉이 들어가면 이야기가 달라진다..! ”



# 전통적인 모델 기반의 한계

- 전통적인 모델 기반의 제어는 **모델, 파라미터, 접촉조건**이 조금만 어긋나도 성능이 급락
- ‘규칙을 사람이 계속 추가’하는 방식은 **확장 비용이 폭발**
- 목표가 ‘정해진 반복 동작’에서 ‘환경 변화에도 되는 범용 조작’으로 옮겨가면서, 학습 기반 제어가 필요해졌다!

- **접촉 및 마찰:**

- : 끼워맞춤, 문 열기, 닦기, 밀기, 비틀기

- **변형 물체**

- : 천, 비닐, 케이블, 스펀지, 음식 등

- **불확실성**

- : 물체 위치 오차, 미끄러짐, 집기 실패, 센서 노이즈

- **긴 지평선 작업**

- : 잡고 → 맞추고 → 조정하고 → 다시 잡고... 같은 다단계 과정



그럼 학습 시키자 !

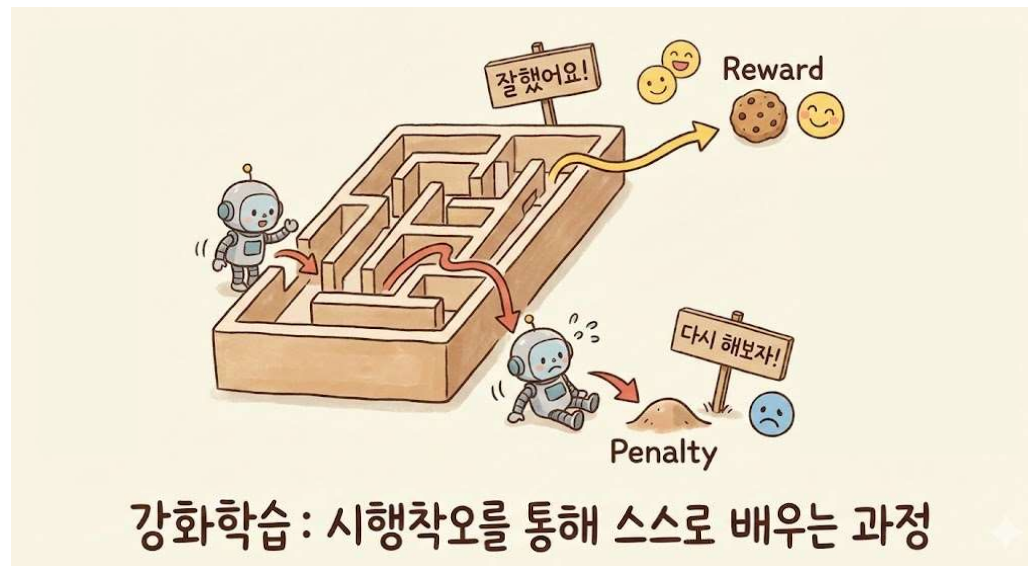
# Learning Robust Autonomous Navigation and Locomotion for Wheeled-Legged Robots

Joonho Lee, Marko Bjelonic, Alexander Reske, Lorenz Wellhausen, Takahiro Miki, and Marco Hutter



# “규칙을 쓰는 대신, 성공을 정의하고 스스로 찾게 한다”

- 강화학습(RL)의 적용 사람이 ‘정답 동작’을 직접 설계하기보다 보상(reward) 으로 성공을 정의  
→ 정책(policy) 이 행동을 학습
- 정해진 해법이 없거나, 손으로 튜닝하기 어려운 제어가 가능
- 특히, 눌러 넣기, 비틀기, 문 열기 등 접촉, 마찰이 있어도, 경험을 통해 견고한 행동을 찾을 수 있다!



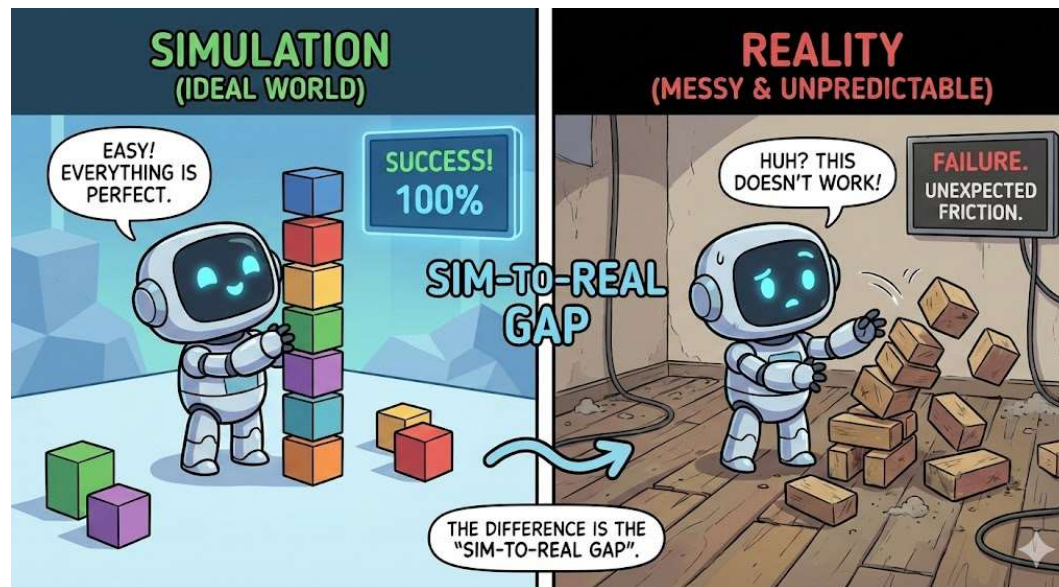


# Multi-Agent Hide and Seek

그러나.. RL의 현실적인 벽이 있으니..

# 강화학습의 한계

- 학습 초기에 로봇이 이상행동을 하며 장비를 망가뜨릴 수 있기 때문에 실물 로봇에서 수만~수백만 번 시도는 불가능
- 시뮬레이션으로만 학습하면?
- 시뮬레이션 공간에서는 잘 돼도 현실에서 미끄러짐, 마찰, 센서가 달라 성능이 깨진다. **Sim-to-Real Gap!**

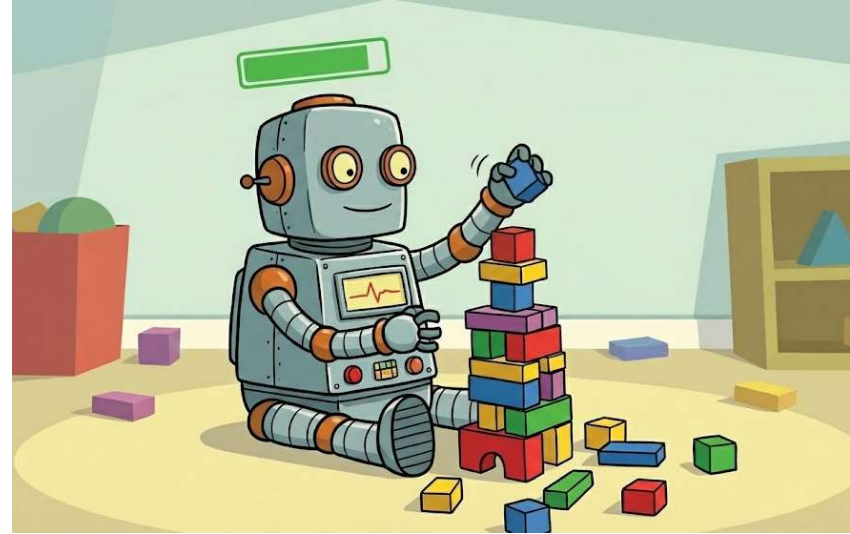
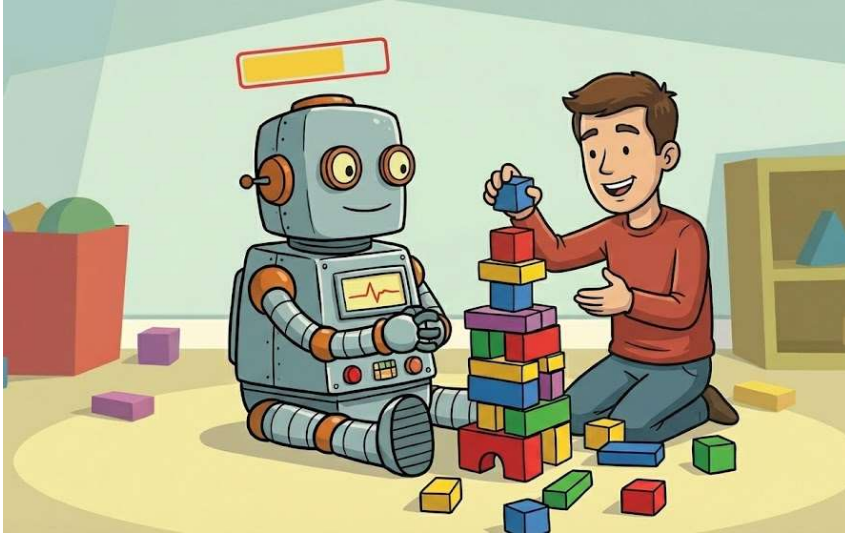


**좋은 행동을 직접 알려줄 방법은 없을까?**

## 모방학습(IL) 도입

- 전문가 시연으로 좋은 초기 행동을 확보
- 학습의 출발점을 ‘엉망인 탐색’이 아니라 ‘그럴듯한 행동’으로 올려준다.

“ 사람이 ‘길’을 보여주면, RL이 훨씬 빨라진다 ! ”



# 모방학습에 필요한 데이터

- 모방학습은 정책( $\pi$ )이 받는 입력(observation)과 내보내는 출력(action)을 정하기 때문에 그에 맞춰 ( $o, a$ ) 시퀀스를 시간 정렬된 에피소드로 수집하는 문제
- 정책이 무엇을 보고(입력) 무엇을 내보낼지(출력)
  - 입력  $o_t$  : 로봇이 지금 무엇을 보고/느끼는가
  - 출력  $a_t$  : 로봇에게 다음에 어떤 명령을 내릴 것인가

## 입력(Observation) 후보

- 비전 (대표)
  - RGB (필수)  
: 작업대, 물체, 손 상태
  - Depth  
: 거리, 형상, 접촉 전후 변화
- 로봇 고유의 감각 정보(Proprioception)
  - 관절각, 속도, 토크(또는 전류)
  - 그리퍼 상태
- 접촉/힘(필요 작업에서 강력)
  - 힘, 토크(FT sensor),  
손끝 촉각(tactile) 등

## 출력(Action) 후보

- 관절 공간
  - joint position
  - joint velocity
  - joint torque
- 작업 공간(EE)
  - end-effector pose
  - twist(6D velocity)
- 그리퍼
  - open/close(이진)
  - 목표 폭(연속)



**사실 모방학습은 여러 종류다..!**

# IL의 큰 두 갈래

## BC(Behavior Cloning)

: 전문가의 행동을 그대로 ‘따라하기’

- BC는 행동 자체를 학습
- 학습 데이터:  $(\mathbf{o}_t, \mathbf{a}_t)$ 
  - 관측  $\mathbf{o}_t$ : 카메라, 관절각 등
  - 행동  $\mathbf{a}_t$ : 관절 목표값, 그리퍼 명령 등

$$\pi = \arg \min_{\pi} \sum_t \|\pi(o_t) - a_t^{expert}\|$$

“이 장면에선 전문가처럼 이렇게 움직여라”

## IRL(Inverse Reinforcement Learning)

: 전문가가 ‘무엇을 원했는지’를 먼저 추정하고,  
그 보상으로 정책을 학습(RL)

- 보상을 학습  $\rightarrow$  그 다음 정책을 학습
- 입력 데이터  
: 전문가 궤적  $\tau = (s_0, s_1, \dots)$  또는  $\mathbf{o}_t$
- 먼저 보상 함수  $r(s, a)$  혹은  $r(s)$  추정
- 그 다음 RL로  $\pi$ 를 최적화

$$r \leftarrow IRL(\tau_{expert}), \quad \pi \leftarrow RL(r)$$

“전문가는 뭘 최대화하려고 이렇게 움직였나?”

## 예시를 통한 BC / IRL 비교

### BC

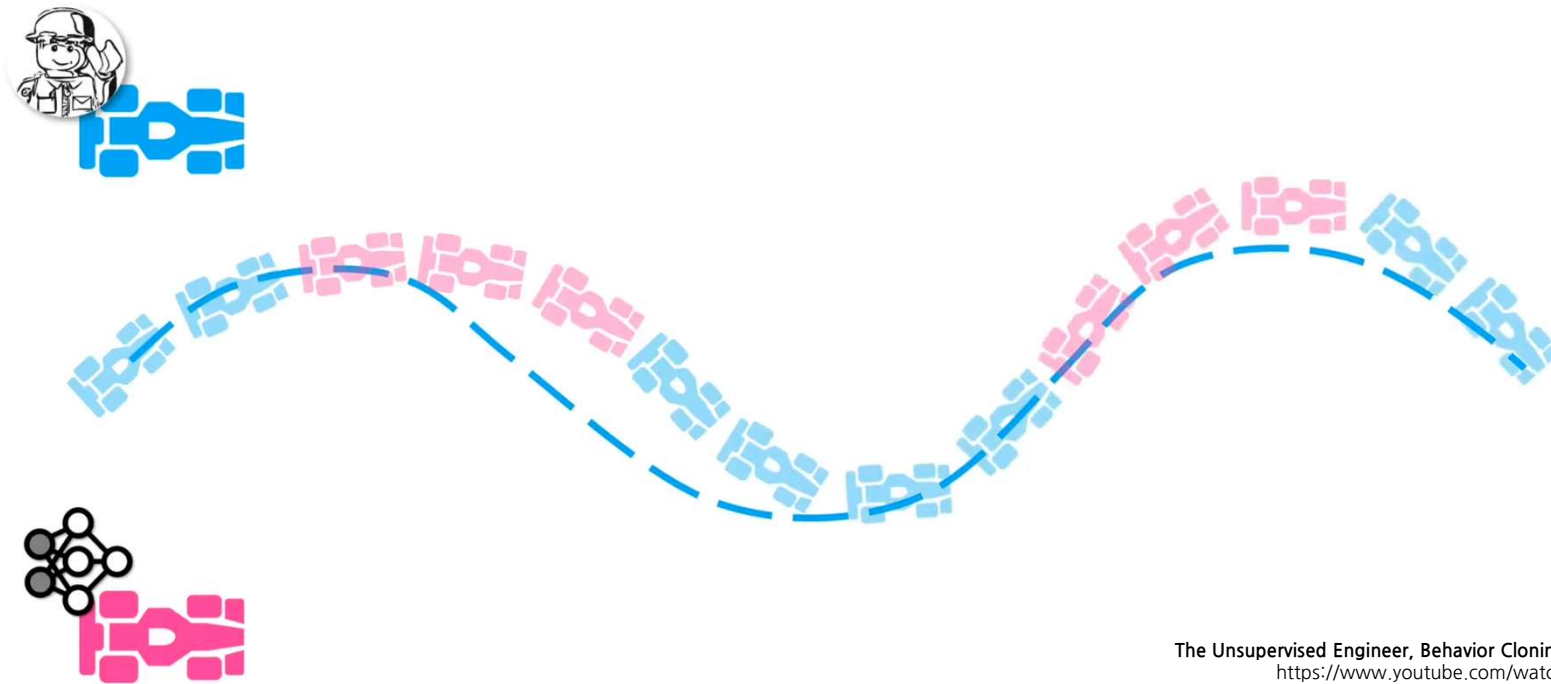
- 데모에서 '손(그리퍼)이 컵에 가서 잡는' 행동을 그대로 학습
- **장점**  
: 구현 쉬움, 데이터만 있으면 바로 학습
- **단점**  
: 컵 위치가 조금 다르거나 미끄러지는 등 데모에서 벗어나면 오류 누적 가능

### IRL

- **데모를 보고 보상을 추정:**
  - 컵에 가까워질수록 보상 ↑
  - 컵을 떨어뜨리면 보상 ↓
  - 그리퍼가 컵을 잡고 안정적으로 들면 보상 ↑
- 그 보상으로 RL을 돌려 정책을 학습
- **장점**  
: 데모와 조금 달라도 '목표(보상)'를 따라가며 새 상황에 적응
- **단점**  
: 보상 추정 + RL까지 필요해서 복잡, 비용 ↑

# 자매품 DAgger

- 실행하면서 데이터 분포를 고치는 방식 (Interactive IL)
- **DAgger**: 정책이 실제로 방문한 상태를 전문가가 다시 라벨링해서 데이터셋을 누적  
→ BC의 오류 누적(분포 이탈) 문제를 줄이는 대표 방법



The Unsupervised Engineer, Behavior Cloning (Part 3): DAgger  
<https://www.youtube.com/watch?v=gffZio5UmQc>

# LeRobot ACT 모델은?

## 입력 $o_t$

- RGB 이미지(1개 이상 카메라)
  - front/top/wrist 카메라 등 “여러 시점의 RGB 이미지”
- 현재 관절각(joint positions)
  - SO-ARM101은 한 팔에 joint 1~6(그리퍼가 joint 6로 표기)로 구성되므로, 관절각 벡터는 보통 6차원(그리퍼 포함)으로 생각

## 출력 $a_t$

- 목표 관절 위치 (6개)
- ‘k 스텝짜리 미래 action chunk’를 한 번에 출력..!

학습 데이터에 사용하는 이미지 구성 예시



TopView Camera Image



Wrist Camera Image

어떻게 알려주지?

# 모방학습의 도구

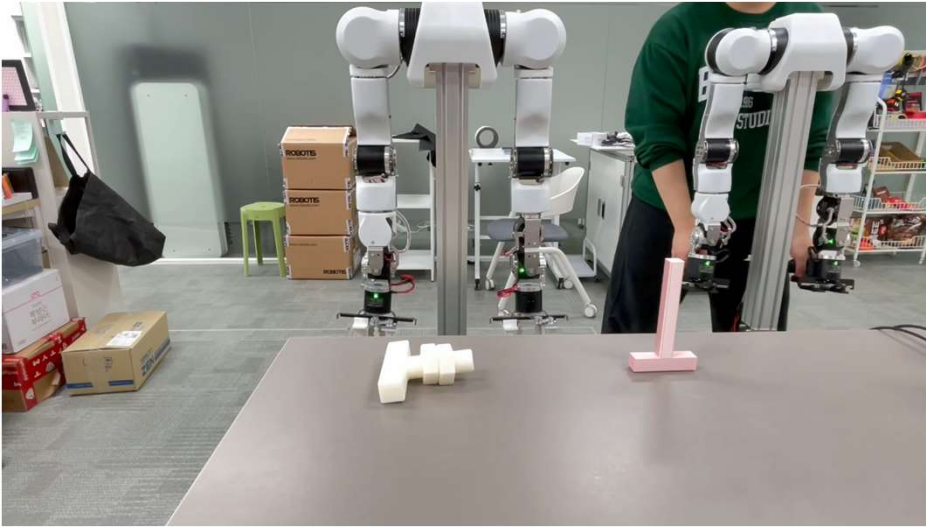
## 시연(Expert) 획득 방식

- Kinesthetic teaching (손으로 잡고 가르치기)
    - 장점: 직관적, 접촉 포함 작업에 강함
    - 단점: 속도 및 타이밍이 들쭉날쭉 → 후처리 중요
  - Scripted expert (기존 제어 및 플래너가 시연 생성)
    - 장점: 대량 생성 가능(시뮬레이션에 특히 좋음)
    - 단점: 사람이 하는 “요령”이 빠질 수 있음
- ★ Teleoperation

  - VR 컨트롤러, 6D 마우스, 조이스틱, 마스터암
  - 장점: 자연스러운 양팔 협응, 데이터 생산 쉬움
  - 라벨(a): 사람이 보낸 명령 자체가 action 레이블이 됨



## 양팔로봇의 경우



양팔로봇 모방학습을 위한 Teleoperation 1.  
형상이 같은 마스터 장비를 사용하는 모습

- **장점**  
로봇이 실제로 움직인 관절/EE 궤적이 곧 '정답 행동'이라서, 동기화/좌표계 변환 이슈가 적음
- **단점**  
힘들다...



양팔로봇 모방학습을 위한 Teleoperation 2.  
웨어러블 장비를 사용하는 모습

- **장점**  
조작자가 자세를 바꾸며 편하게 수행할 수 있고, 여러 로봇, 여러 세션에 확장하기 좋음
- **단점**  
컨트롤러 좌표(사람) → 로봇 좌표(월드/베이스/EE) 변환, 스케일, 오프셋이 들어지면 데이터 품질이 급락



# LeRobot

- 로봇 모방학습(Imitation Learning) 워크플로우를 빠르게 돌리기 위한 오픈소스 툴킷
- 로봇 모방학습은 보통 다음이 번거로움
  - 카메라/관절 데이터 동기화
  - 리플레이/정제(데이터셋 만들기)
  - 학습 코드와 실행 코드 분리(재현 어려움)
- 수집-학습-실행을 한 파이프라인으로 표준화
- ‘일단 돌아가는 데모’를 빠르게 만들고, 반복 개선하기 쉬움
- <https://huggingface.co/>

# LeRobot 사용 준비

[https://github.com/pinklab-art/rosconkr\\_il](https://github.com/pinklab-art/rosconkr_il)

## Installation

LeRobot 및 하드웨어 제어 관련 의존성을 설치합니다.

```
# 가상환경 생성
mkdir -p ~/venv && cd ~/venv
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install python3.10 python3.10-venv python3.10-dev -y

python3.10 -m venv lerobot

# 가상환경 활성화
source ~/venv/lerobot/bin/activate

# ffmpeg 설치
sudo apt update
sudo apt install ffmpeg

# lerobot git clone
git clone https://github.com/huggingface/lerobot.git
cd ~/lerobot

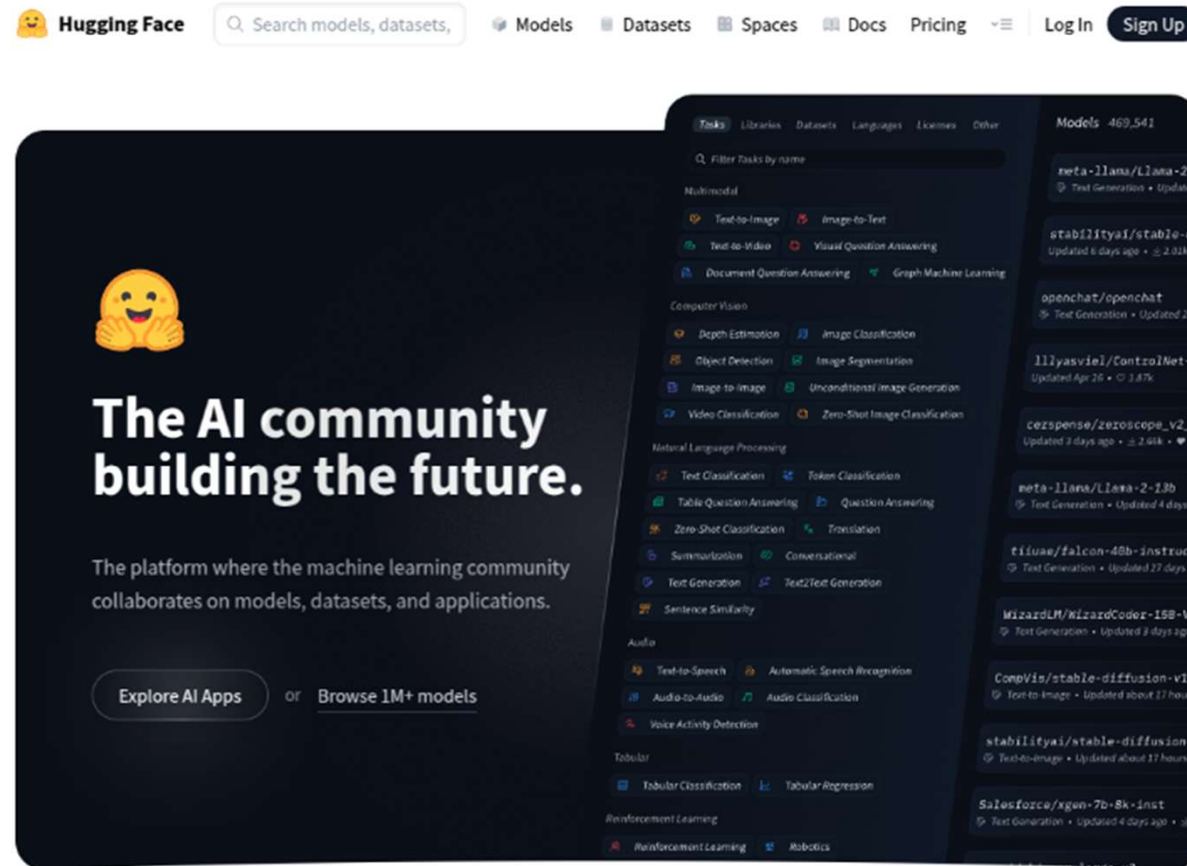
# 패키지 설치
pip install -e .

# 추가 기능
pip install -e ".[all]"           # 모든 기능 설치
pip install -e ".[aloha,pusht]"  # (Aloha 및 Pusht)
pip install -e ".[feetech]"      # Feetech motor(so-101 사용시 설치)
pip install -e ".[dynamixel]"   # Dynamixel motor
```



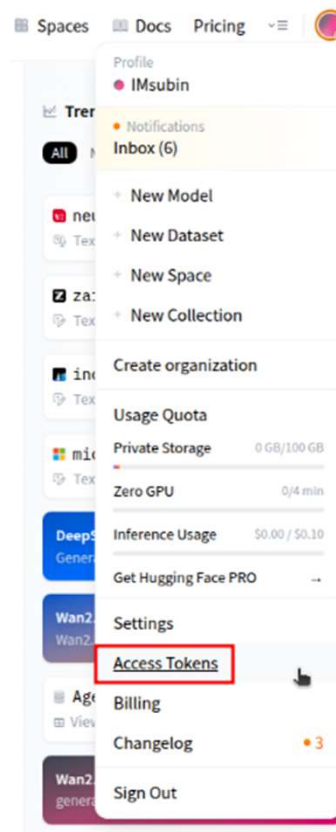
# Hugging Face 사용 준비 (01/07)

허깅페이스 회원가입 - [링크](#)



# Hugging Face 사용 준비 (02/07)

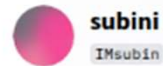
Access Tokens 클릭



# Hugging Face 사용 준비 (03/07)

Create new token

Comment



Profile

Account

Authentication

Organizations

Billing

Access Tokens

## Access Tokens

### User Access Tokens

+ Create new token

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. **Do not share your Access Tokens with anyone;** we regularly check for leaked Access Tokens and remove them immediately.

You have no Access Token

# Hugging Face 사용 준비 (04/07)

## 토큰 생성

- Write로 생성

### < Create new Access Token

#### Token type

Fine-grained

Read

**Write**

ⓘ This cannot be changed after token creation.

#### Token name

IL

This token has read and write access to all your and your orgs resources and can make calls to Inference Providers on your behalf.

Create token

# Hugging Face 사용 준비 (05/07)

토큰 복사하여 잘 저장해두기

 Save your Access Token

Save your token value somewhere safe. **You will not be able to see it again after you close this modal.** If you lose it, you'll have to create a new one.

 Copy

| Name | Permissions      |
|------|------------------|
| IL   | <div>WRITE</div> |

Done

# Hugging Face 사용 준비 (06/07)

## 토큰 등록

- \${HUGGINGFACE\_TOKEN}를 부분을 지워 수정

```
1 cd ~/lerobot
2 git config credential.helper store
3 hf auth login --token ${HUGGINGFACE_TOKEN} --add-to-git-credential
```

```
Token is valid (permission: write).
The token `IL` has been saved to /home/subin/.cache/huggingface/stored_tokens
Your token has been saved in your configured git credential helpers (store).
Your token has been saved to /home/subin/.cache/huggingface/token
Login successful.
The current active token is: `IL`
```



# Hugging Face 사용 준비 (07/07)

현재 로그인된 Hugging Face 계정의 사용자명을 쉘 변수로 저장

```
1 HF_USER=$(hf auth whoami | head -n 1 | sed 's/\x1b\[[0-9;]*m//g' | awk -F': ' '{print $2}')
```

```
(lerobot) subin@subin-desktop:~/lerobot$ HF_USER=$(hf auth whoami | head -n 1 |  
sed 's/\x1b\[[0-9;]*m//g' | awk -F': ' '{print $2}' | xargs)  
(lerobot) subin@subin-desktop:~/lerobot$ echo $HF_USER  
IMsubin  
(lerobot) subin@subin-desktop:~/lerobot$
```

# SO101 모방학습 1. Data Collection

## 1. Data Collection (데이터 수집)

텔레오퍼레이션(Teleoperation)을 통해 로봇의 상태(State)와 이미지(Image) 데이터를 수집합니다.

### Robot Setting

- [OpenManipulator-X](#)
- [SO-101](#)

각 링크를 참조하여 로봇 세팅을 완료해주세요.

### Teleoperation Example

#### Command

```
lerobot-teleoperate \  
  --robot.type=so101_follower \  
  --robot.port=/dev/ttyACM0 \  
  --robot.id=follower \  
  --teleop.type=so101_leader \  
  --teleop.port=/dev/ttyACM1 \  
  --teleop.id=leader \  
  --robot.cameras="{  
    top: { type: opencv, index_or_path: /dev/video0, width: 640, height: 480, fps: 30 },  
    wrist: { type: opencv, index_or_path: /dev/video2, width: 640, height: 480, fps: 30 }  
  }"
```



# SO101 모방학습 2. Review & Train

## 2. Demonstration Review & Train (데모 리뷰 및 학습)

수집된 데이터의 품질을 검증하고 정책(Policy) 모델을 학습합니다.

### 데이터 시각화 및 검수

- 관찰과 액션 값의 동기화가 잘 되었는지 점검
- 수집 시 액션이 부드러운지 점검
- 조명 혹은 블러가 되었는지 점검

### Replay Dataset

```
lerobot-replay \  
  --robot.type=so101_follower \  
  --robot.port=/dev/tty.usbmodem58760431541 \  
  --robot.id=my_awesome_follower_arm \  
  --dataset.repo_id=${HF_USER}/record-test \  
  --dataset.episode=0
```



## 모델 학습 (ACT / SmoIVLA)

데이터셋을 사용하여 모델을 학습시킵니다.

### Option A: ACT (Action Chunking Transformer)

```
lerobot-train \  
  --dataset.repo_id=${HF_USER}/your_dataset \  
  --policy.type=act \  
  --output_dir=outputs/train/act_your_dataset \  
  --job_name=act_your_dataset \  
  --policy.device=cuda \  
  --wandb.enable=true \  
  --policy.repo_id=${HF_USER}/act_policy
```

# SO101 모방학습 3. Inference

## 3. Inference (추론)

학습된 모델을 로봇에 배포하여 추론을 수행합니다. 기본 `eval.py` 스크립트는 동기식(Synchronous)으로 동작합니다.

특징:

- `Observation` → `Inference` → `Action` 과정이 순차적으로 실행됩니다.
- 추론 연산 시간 동안 로봇의 제어 루프가 대기(Block)하게 되어, 모델이 무거울수록 제어 주기가 불안정해질 수 있습니다.

Command:

```
lerobot-record \
  --robot.type=so100_follower \
  --robot.port=/dev/ttyACM0 \
  --robot.id=my_robot \
  --robot.cameras="{ front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}}" \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/eval_act_your_dataset \
  --dataset.num_episodes=10 \
  --dataset.single_task="Your task description" \
  --policy.path=${HF_USER}/act_policy
```



# SO101 모방학습 4. Asynchronous Inference

(Policy)Server Start Command:

```
python -m lerobot.async_inference.policy_server \
  --host=127.0.0.1 \
  --port=8080 \
  --fps=25 \
  --inference_latency=0.033 \
  --obs_queue_timeout=1
```

(RoBot)Client Start Command:

```
python -m lerobot.async_inference.robot_client \
  --robot.type=so101_follower \
  --robot.port=/dev/so101_follower \
  --robot.id=follower \
  --robot.cameras='{ \
    top: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 25}, \
    wrist: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 25}, \
  }' \
  --task=IMsubin/pick_tomato_place_pot \
  --server_address=127.0.0.1:8080 \
  --policy_type=act \
  --pretrained_name_or_path=IMsubin/pick_tomato_place_pot \
  --policy_device=cuda \
  --actions_per_chunk=100 \
  --chunk_size_threshold=0.5 \
  --aggregate_fn_name=weighted_average \
  --debug_visualize_queue_size=True
```

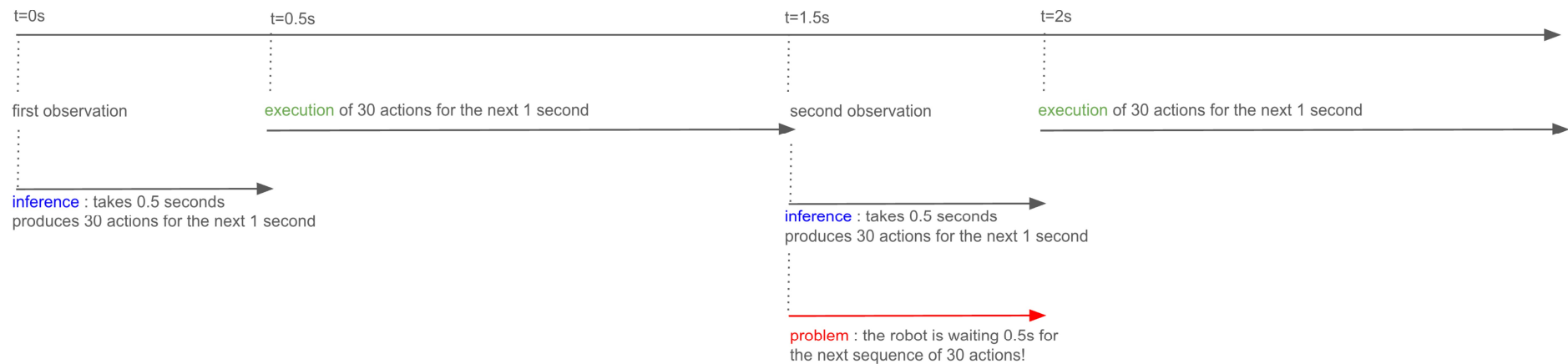


비동기 추론이란?

# 기존 추론의 문제점

기존의 추론 방식은 비효율적이다.

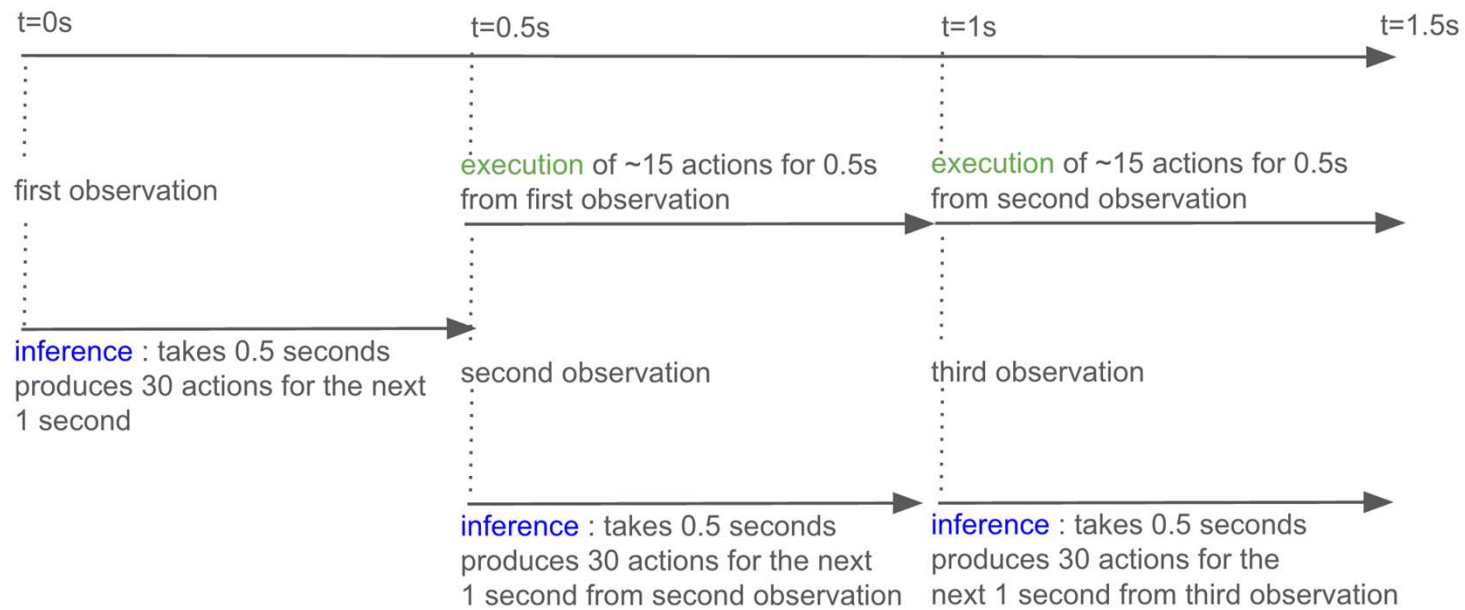
**Synchronous inference:** planning and execution are interleaved, causing idleness



# 비동기 추론 방식

비동기 추론을 사용해서 끊임없이 제어해보자!

## Asynchronous implementation





# 글로 자세히..

## 타임라인

### 1. $t = 0.00s$

- RobotClient가 관측 o0(카메라+관절상태)를 서버로 보냄

### 2. $t = 0.25s$

- 서버가 ACT로 청크 A0..A49(50개 액션)을 만들어 RobotClient로 보냄
- RobotClient는 큐에 50개를 넣고 실행 시작

### 3. $t = 0.25s \sim 1.50s$

- 20Hz로 25개 실행하면( $25 \times 0.05 = 1.25초$ ) 큐에 **25개 남음**
- 남은 비율이 0.5 수준이므로, RobotClient는 **관측 o1을 서버로 미리 전송** (“다음 청크 미리 계산해줘”)

### 4. $t = 1.75s$ (요청 후 0.25초 경과)

- 그 사이 로봇은 멈추지 않고  $0.25초 / 0.05초 = 5개$  액션을 더 실행
- 원래 남았던 25개 중 5개를 써서 **20개 남은 상태**에서, 서버로부터 새 청크 B0..B49 도착

### 5. 청크 “겹치는 구간”을 섞어서 갱신(aggregate)

- B00..B04는 추론하는 동안 실행된 A25..A29와 겹치는 부분이므로 버림
- 기존에 남아있던 20개(A30..A49)와 새 청크의 앞 20개(B05..B24)는 **시간상 같은 “다가올 1.0초( $=20 \times 0.05$ )” 구간**을 말하므로 겹침
- LeRobot은 이 겹치는 부분을 aggregate\_fn\_name(예: weighted\_average)로 섞고, 뒤쪽은 새 청크를 이어 붙여 큐를 업데이트

```
지금스텝 = now_step()
```

```
새청크 = server로부터 받음 # (액션들 + 그 액션들이 대응되는 timestep 정보)
```

```
버릴개수 = 지금스텝 - 새청크.시작timestep
```

```
만약 버릴개수 > 0:
```

```
    새청크 = 새청크에서 앞의 버릴개수 만큼 제거
```

```
겹치는길이 = min(현재큐.남은개수, 새청크.개수)
```

```
겹치는구간은(현재큐, 새청크)를 선택한 방식(weighted_average 등)으로 섞음
```

```
나머지 새청크는 현재큐 뒤에 이어붙임
```

여기서 끝이 아니다~!

# 실전 적용 1: 멀티-모델 오케스트레이션

“상황에 따라 ‘맞는 정책’을 선택한다”

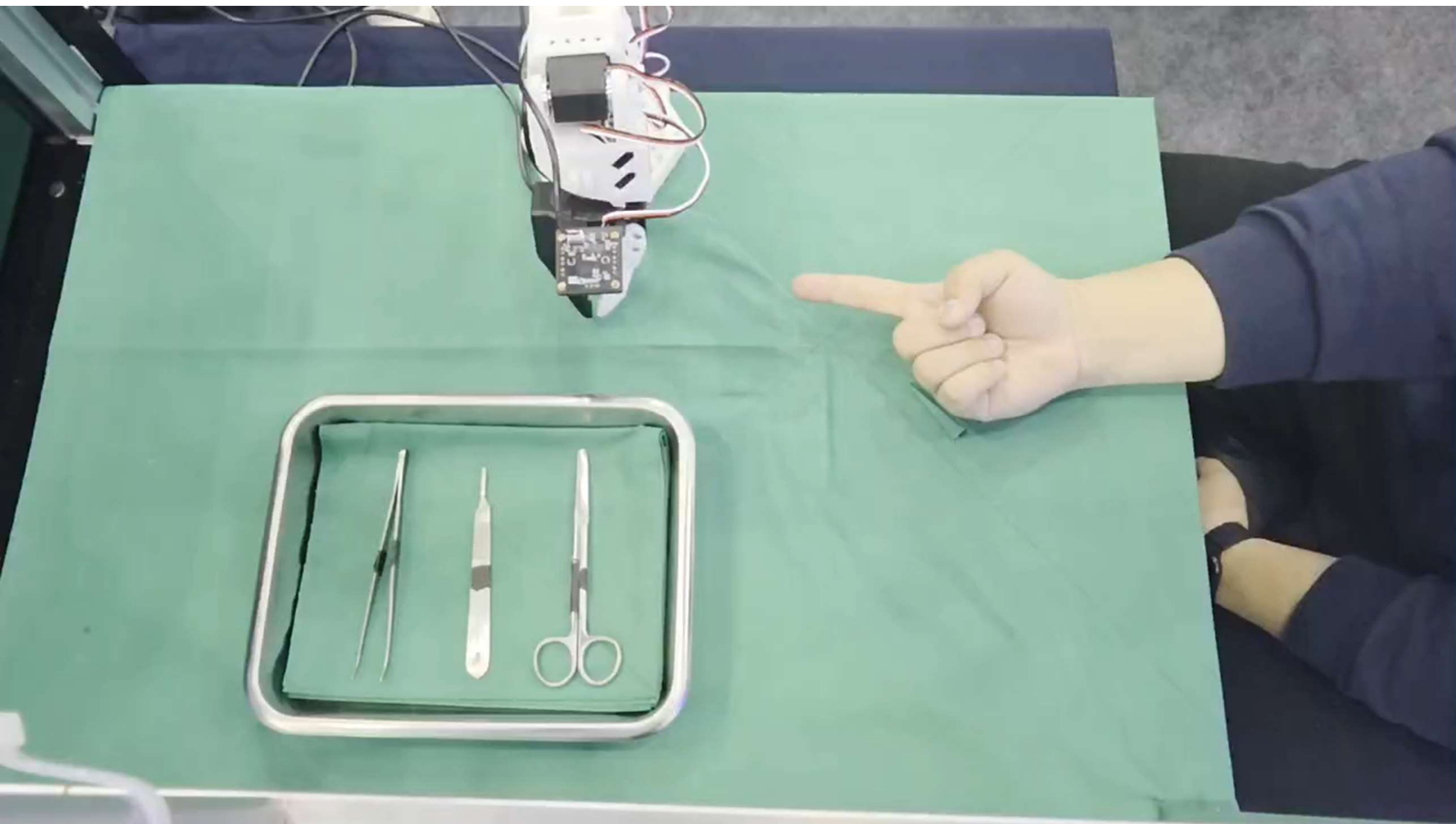
실제 작업에서는 정책 하나로 끝내기보다

작업을 상황으로 나누어 각 구간에 특화된 여러 정책(모델)을 학습·준비해두고,

선택(Select) → 실행(Act) → 전환(Switch) 하는 오케스트레이션이 성능과 운영성을 동시에 향상시킬 수 있다.

왜 ‘멀티-모델’이 실전에서 유리할까?

- 작업이 바뀔 때마다 단일 정책을 다시 학습시키면 비용이 크다.
  - 재사용 가능한 정책 라이브러리가 유리
- 서로 다른 물체/접촉 양상은 최적 행동이 다르다.
  - ‘컵 집기 정책’과 ‘서랍 손잡이 돌리기 정책’은 입력 및 행동 분포가 달라 분리할수록 안정
- 실패 시 복구 전략이 다르다.
  - ‘재시도(regrasp) 정책’, ‘정렬 복구 정책’처럼 리커버리 전용 정책을 따로 두면 운영이 쉬움



# ‘물체를 보고, 그 물체에 맞는 정책을 골라 쓰는’ 의사코드 예시

준비(시작할 때 한 번):

- 컵용 AI, 병용 AI, 서랍손잡이용 AI ... 필요한 AI들을 미리 전부 불러오고
- ‘어떤 상황 + 어떤 물체면 어떤 AI를 쓸지’ 규칙표를 만들어 둔다

반복(작업 중 계속):

- 1) 카메라/로봇 상태를 읽는다
- 2) 지금 보이는 물체가 뭔지 인식 (예: 컵, 병, 서랍손잡이...)
- 3) 지금 단계가 뭔지 정한다 (예: 찾는 중 / 다가가는 중 / 잡는 중 / 조작 중)
- 4) 규칙표를 보고,
  - (지금 단계, 물체 종류)에 맞는 AI가 있으면:
    - 그 AI를 실행해서 "다음에 이렇게 움직여"라는 제안을 받는다
    - 그 제안을 다음 단계(안전 검사)로 넘긴다
  - 없으면:
    - 기본 동작(천천히 이동/대기/정해진 동작)을 한다

**안전제일!**

## 실전 적용 2: 정책 출력 검증(게이트)

### “안전한 명령으로 변환해 실행하기”

실제 실행 명령  $a$  는 검증, 제한, 완화(필터링)를 거쳐 생성해야 한다.

$$\hat{a}_t = \pi(o_t) \rightarrow \text{Gate}(\hat{a}_t, o_t, x_t) \rightarrow a_t \rightarrow \text{Execute}$$

$o_t$ : observation(카메라, joint 등)

$x_t$ : 로봇 상태/추정(관절각, 접촉 추정 등)

Gate: 안전/품질 검증 + 제한 + 풀백

### Gate의 체크리스트 예시

- 액션 유효성 체크(Action sanity)
  - NaN/Inf 검사
  - “너무 큰 점프” 검사:  $\|\Delta a\| > \text{limit}$
  - 모델이 내면 안 되는 범위: 관절/그리퍼 물리 범위
  - 실패 시: 클리핑/홀드/풀백
- 제약 적용(Constraints & limiting)
  - 관절 범위 클리핑:  $q_{\min} \leq q_{\text{cmd}} \leq q_{\max}$
  - 속도 제한( $\Delta q$  제한):  $|q_{\text{cmd}} - q_{\text{curr}}| \leq dq_{\max\_per\_step}$
  - 평활화(smoothing): EMA/low-pass로 떨림 감소
- 입력 품질 체크(Observation validity)
  - 카메라 프레임 최신성:  $\text{now} - \text{img\_stamp} < 100\text{ms}$
  - 타겟 인식 신뢰도:  $\text{conf} > \tau$
  - TF/좌표 유효성(필요 시): 프레임 누락/점프 감지
  - 실패 시: 정책 OFF 또는 이전 안정 명령 유지(hold)
- 실행/모니터링(Execution monitor)
  - 오차 폭증(추종 안 됨)
  - 접촉 추정 과도(토크/전류 급증)
  - 관측 끊김/프레임 드랍 지속
  - 이상 발생 시 즉시: `policy_off + safe_stop/retreat` 상태 전이